LEARN BEING FORWARD

Programming — Java Programming

# Multithreading basics

COMPLETE LEARNING GUIDE

### What You'll Learn

Modern hardware has multiple CPU cores, and taking advantage of them means running independent pieces of work concurrently rather than one instruction at a time — but concurrency also introduces an entirely new category of bugs (race conditions, deadlocks) that don't exist in single-threaded code and can be maddeningly hard to reproduce. This sub-module starts from the basics of what a thread is and how to create one, builds a hands-on understanding of race conditions by deliberately triggering one, then fixes it with the synchronized keyword, and moves up to the higher-level, safer tools most production code actually uses day to day — ExecutorService thread pools and the atomic classes — closing with a brief, practical introduction to deadlock so you can recognize and avoid the classic circular-wait pattern.

## 1. Processes, Threads, and Why Concurrency Matters

A process is an independent running program with its own isolated memory space; a thread is a lightweight unit of execution within a process that shares that process's memory with every other thread in it — which is exactly what makes threads fast to create and communicate between, and exactly what makes shared mutable state dangerous. Every Java program already runs on at least one thread (the 'main' thread), and multithreading means launching additional threads that run concurrently, letting a program do productive work (handling another request, running a background computation) while one thread waits on something slow like disk or network I/O.

## 2. Creating Threads: extends Thread vs implements Runnable

A thread's task can be defined by extending Thread and overriding its run() method, or — the generally preferred approach — by implementing the Runnable functional interface and passing an instance to a plain new Thread(runnable). Implementing Runnable is preferred because Java only allows single inheritance: extending Thread uses up your one available superclass slot, while implementing Runnable leaves that slot free and also lets the same task be reused with other concurrency tools (like an ExecutorService) that expect a Runnable rather than a Thread subclass.

```java
Runnable task = () -> {
    System.out.println("Running on: " + Thread.currentThread().getName());
};

Thread t = new Thread(task, "worker-1");
t.start(); // starts a new thread — NOT t.run(), which would run synchronously
```

## 3. Thread Lifecycle and Core Methods

Calling start() transitions a thread from NEW to RUNNABLE and schedules it to actually execute concurrently — calling run() directly instead executes the code synchronously on the calling thread, with no concurrency at all, which is one of the very first mistakes almost every learner makes. Thread.sleep(ms) pauses the current thread for at least the given duration without consuming CPU; join() blocks the calling thread until the target thread finishes, which is essential when the rest of the program depends on a background thread's result being ready. Both sleep() and join() can throw the checked InterruptedException, which should be handled deliberately rather than swallowed silently.

```java
Thread worker = new Thread(() -> {
    try {
        Thread.sleep(1000);
        System.out.println("Worker finished");
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt(); // restore the interrupt status
    }
});

worker.start();
worker.join(); // main thread waits here until worker finishes
System.out.println("Main thread continues after worker");
```

## 4. Race Conditions and Shared Mutable State

A race condition occurs when two or more threads read and write shared mutable state without coordination, and the final result depends on the unpredictable timing of their interleaving. Incrementing a shared counter looks like one operation in source code, but is actually three separate steps at the machine level — read the value, add one, write it back — and if two threads each read the same value before either writes back, one increment is silently lost. This is exactly why a seemingly simple counter incremented concurrently by multiple threads often ends up lower than expected, and why the bug may not even reproduce reliably in testing.

```java
class Counter {
    private int count = 0;
    public void increment() { count++; } // NOT atomic: read, add, write
    public int getCount() { return count; }
}

// With 1000 threads each calling increment() once, the final count
// is frequently LESS than 1000 due to lost updates from interleaving.
```

## 5. The synchronized Keyword and Intrinsic Locks

Marking a method or block synchronized ensures only one thread at a time can execute that code while holding the associated object's intrinsic lock — any other thread attempting to enter must wait until the lock is released. Applied consistently to every method that reads or writes the same shared state, this eliminates the interleaving that causes race conditions, at the cost of serializing access (which is a deliberate performance tradeoff for correctness). A common and subtle mistake is synchronizing on different lock objects in different methods that touch the same shared state — since locks only provide mutual exclusion against other threads trying to acquire that exact same lock object, mismatched locks provide no real protection at all.

```java
class Counter {
    private int count = 0;
    public synchronized void increment() { count++; } // now atomic w.r.t. other synchronized calls
    public synchronized int getCount() { return count; }
}
```

## 6. Higher-Level Concurrency: ExecutorService and Thread Pools

Creating a brand-new Thread for every single task is wasteful — thread creation and teardown carry real overhead, and an unbounded number of threads can exhaust system resources. The java.util.concurrent.ExecutorService abstracts task execution behind a managed pool of reusable worker threads: you submit Runnable or Callable tasks, and the pool handles scheduling them across its fixed set of threads, reusing them rather than constantly creating new ones. This is the standard, production-grade way to run concurrent work in Java, and it must be explicitly shut down (shutdown() or shutdownNow()) when no longer needed, or its threads will keep the JVM from exiting.

```java
import java.util.concurrent.*;

ExecutorService pool = Executors.newFixedThreadPool(4);

for (int i = 0; i < 10; i++) {
    int taskId = i;
    pool.submit(() -> System.out.println("Task " + taskId + " on " +
Thread.currentThread().getName()));
}

pool.shutdown(); // stop accepting new tasks; lets submitted ones finish first
```

## 7. volatile and the Atomic Classes

The volatile keyword guarantees that reads and writes to a field go directly to main memory rather than being cached per-thread, so every thread always sees the most recently written value — but it only guarantees visibility, not atomicity, so volatile int count; count++; is still unsafe for concurrent increments since the increment itself remains three separate steps. The java.util.concurrent.atomic package (AtomicInteger, AtomicLong, AtomicReference) provides genuinely atomic compound operations like incrementAndGet() using efficient lock-free hardware instructions, making it the right tool for simple shared counters without the overhead of a full synchronized block.

```java
import java.util.concurrent.atomic.AtomicInteger;

AtomicInteger counter = new AtomicInteger(0);
counter.incrementAndGet(); // atomic — safe to call from many threads
concurrently
System.out.println(counter.get());
```

## 8. Avoiding Deadlock: A Brief Introduction

Deadlock happens when two or more threads each hold a lock the other needs and neither will ever release its own — a circular wait with no possible resolution, freezing the affected threads permanently. The classic case is two threads locking the same two resources in opposite order (thread A locks resource 1 then waits for resource 2, while thread B has already locked resource 2 and is waiting for resource 1). The most reliable prevention is a simple discipline: always acquire multiple locks in a single, consistent, agreed-upon order throughout the entire codebase, which makes the circular-wait pattern structurally impossible.

### Common Pitfalls

- Calling thread.run() directly instead of thread.start(), which executes the code synchronously on the calling thread with no actual concurrency.

- Forgetting to synchronize access to shared mutable state accessed by multiple threads, producing race conditions that may pass in testing and fail unpredictably in production.
- Synchronizing on different lock objects in different methods that touch the same shared data, providing no real mutual exclusion at all.
- Assuming volatile alone makes a compound operation like count++ thread-safe — it only guarantees visibility, not atomicity.
- Creating a new Thread for every task instead of reusing a pooled ExecutorService, incurring unnecessary creation overhead at scale.
- Forgetting to call shutdown() on an ExecutorService, leaving its pool threads alive and preventing the JVM from exiting cleanly.
- Swallowing InterruptedException silently instead of restoring the interrupt status or propagating it, which can hide a legitimate cancellation request.

## Key Takeaways

- Threads within the same process share memory — that's what makes them fast but also what makes shared mutable state dangerous.
- Always call start(), never run() directly, to actually run code on a separate thread.
- A race condition arises when multiple threads read-modify-write shared state without coordination — synchronized (or an atomic class) fixes it.
- synchronized only protects against other threads trying to acquire that exact same lock object — mismatched locks provide no protection.
- Prefer a managed ExecutorService thread pool over manually creating threads for every task.
- volatile guarantees visibility across threads, not atomicity — use the java.util.concurrent.atomic classes for safe compound operations like increments.
- Prevent deadlock by always acquiring multiple locks in the same, consistent order everywhere in the codebase.

## Learn More

**Oracle — The Java Tutorials: Concurrency**
https://docs.oracle.com/javase/tutorial/essential/concurrency/index.html

**Oracle Java 17 API — java.util.concurrent**
https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/package-summary.html

**Baeldung — Guide to the Java ExecutorService**
https://www.baeldung.com/java-executor-service-tutorial